

# **CSE 220 — Convolution Online**

Practice Problem Bank with Worked Solutions

Syntax cheat sheet · 17 problems · every solution executed and verified

Built from the Offline 1 specification and the July 2025 A/B/C online papers

# How to use this pack

---

Every online paper so far has the same shape: you are handed a template that already contains `DiscreteSignal`, `LTISystem` and a `main` block, and you must add one new idea on top — a new signal operation, a new way of combining systems, or a new class. Nothing asks you to rewrite convolution.

So the fastest route through the exam is:

1. Paste in your corrected classes (Part 1) if the template's copy is a stub.
2. Find the closest problem in Part 3 and adapt its solution.
3. Use Part 2 for file reading, plotting and verification syntax you always forget.
4. Part 4 covers the older application-style papers (moving averages, smoothing, polynomials).
5. Run the trap checklist on the last page before you submit.

**Difficulty labels.** **Warm-up** = signal operations only, no convolution. **Core** = one convolution idea, matches the style of the A/B/C papers exactly. **Stretch** = a new class or a non-obvious algorithm; likely the 10-mark question if your paper is harder than last year's.

## Part 1 — Base classes (copy-paste this first)

This is the corrected Offline 1 code. Two things differ from the raw template: `get_response_components` returns `(k, component)` pairs and `get_contributions_at_time` returns 4-tuples `(k, x_k, h_n_minus_k, product)`. If the exam template gives you its own copy, use theirs — but check the return types match what the question expects.

```
import numpy as np

def readable_time_ticks(time_values, max_labels=18):
    if len(time_values) <= max_labels:
        return time_values

    step = int(np.ceil(len(time_values) / max_labels))
    ticks = time_values[::step]

    if ticks[-1] != time_values[-1]:
        ticks.append(time_values[-1])

    return ticks

class DiscreteSignal:
    """Finite discrete-time signal with integer indices."""

    # Create a finite discrete-time signal over the given integer range.
    def __init__(self, start_time, end_time):
        self.start_time=start_time
        self.end_time=end_time
        self.values=np.zeros(end_time-start_time+1)

    # Return the number of stored samples in the signal.
    def __len__(self):
        return len(self.values)

    # Return the integer time indices covered by the signal.
    def times(self):
        return range(self.start_time ,self.end_time+1)

    # Return the signal value at the given time index.
    def get_value_at_time(self, t):
        if (self.start_time <= t <= self.end_time): return self.values[t-self.start_time]
        return 0.0

    # Set the signal value at the given time index.
    def set_value_at_time(self, t, value):
        if (self.start_time <= t <= self.end_time): self.values[t-self.start_time]= float(value)
        else: raise IndexError(f"Time index {t} out of bounds [{self.start_time} , {self.end_time}]")

    # Return a shifted copy of the signal.
    def shift(self, k):
        new_signal = DiscreteSignal(self.start_time+k, self.end_time+k)
        new_signal.values = self.values.copy()
        return new_signal

    # Return the sum of this signal and another signal.
    def add(self, other):
        start = min(self.start_time, other.start_time)
        end= max(self.end_time, other.end_time)
        new_signal = DiscreteSignal(start,end)
        for t in new_signal.times():
            new_signal.set_value_at_time(t,self.get_value_at_time(t)+other.get_value_at_time(t))
        return new_signal

    # Return a scaled copy of the signal.
    def multiply(self, scalar):
        new_signal = DiscreteSignal(self.start_time, self.end_time)
        new_signal.values=self.values*float(scalar)
        return new_signal

    # Return the nonzero samples of the signal.
    def nonzero_samples(self, tolerance=1e-12):
        return [(t,val) for t,val in zip(self.times(),self.values) if abs(val)>tolerance]

    def plot(self, title, save_path=None, ax=None):
        import matplotlib.pyplot as plt

        if ax is None:
            _, ax = plt.subplots()

        time_values = list(self.times())
        markerline, stemlines, baseline = ax.stem(time_values, self.values)
        markerline.set_markersize(6)
        baseline.set_color("black")
        baseline.set_linewidth(1)

        ax.axhline(0, color="black", linewidth=0.8)
        ax.set_title(title)
        ax.set_xlabel("n")
        ax.set_ylabel("value")
        ax.grid(True, alpha=0.35)
        ax.set_xticks(readable_time_ticks(time_values))
        ax.tick_params(axis="x", labelsize=9)

        if save_path is not None:
            plt.savefig(save_path, bbox_inches="tight", dpi=150)

        return ax
```

```

class LTISystem:
    """Discrete-time LTI system described by a finite impulse response."""

    # Store the impulse response that defines the LTI system.
    def __init__(self, impulse_response):
        self.h=impulse_response

    # Return the output time range for the convolution result.
    def output_range(self, input_signal):
        start = self.h.start_time + input_signal.start_time
        end = self.h.end_time + input_signal.end_time
        return start,end

    # Arguments: input_signal is a DiscreteSignal representing x[n].
    # Returns: list of (k, component_signal) for each nonzero input sample x[k].
    # Example: x[2] = 3 contributes the component 3*h[n - 2].
    def get_response_components(self, input_signal):
        components = []
        for k, x_k in input_signal.nonzero_samples():
            shifted_h = self.h.shift(k)
            scaled_shifted_h = shifted_h.multiply(x_k)
            components.append((k, scaled_shifted_h))
        return components

    # Return the system output using superposition of response components.
    def output_by_superposition(self, input_signal):
        out_start, out_end = self.output_range(input_signal)
        y = DiscreteSignal(out_start, out_end)

        components = self.get_response_components(input_signal)
        for k, component in components:
            y = y.add(component)

        return y

    # Arguments: input_signal is a DiscreteSignal and n is one output time index.
    # Returns: list of (k, x_k, h_n_minus_k, product) nonzero contribution tuples.
    # Example: a term may look like (2, 3.0, 0.5, 1.5) for x[2]h[n - 2].
    def get_contributions_at_time(self, input_signal, n):
        contributions = []
        for k in input_signal.times():
            x_k = float(input_signal.get_value_at_time(k))
            if abs(x_k) > 1e-12:
                h_nk = float(self.h.get_value_at_time(n - k))
                term = x_k * h_nk
                if abs(term) > 1e-12:
                    contributions.append((k, x_k, h_nk, term))
        return contributions

    # Arguments: input_signal is a DiscreteSignal and n is one output time index.
    # Returns: float, the convolution-sum value y[n].
    def output_at_time(self, input_signal, n):
        contributions = self.get_contributions_at_time(input_signal, n)
        return float(sum(term for _, _, _, term in contributions))

    # Return the complete output signal of the LTI system.
    def output(self, input_signal):
        out_start, out_end = self.output_range(input_signal)
        y = DiscreteSignal(out_start, out_end)

        for n in y.times():
            y.set_value_at_time(n, self.output_at_time(input_signal, n))

        return y

```

## Part 2 — Syntax cheat sheet

### 2.1 Reading a signal from a file

Format used in every paper so far — line 1 is the range, line 2 is the samples:

```
# file contents:
# -2 3
# 1 0 2 -1 0 3

def read_signal(filename):
    with open(filename, "r", encoding="utf-8") as f:
        lines = [ln.strip() for ln in f if ln.strip()]
    start_time, end_time = map(int, lines[0].split())
    values = list(map(float, lines[1].split()))
    assert len(values) == end_time - start_time + 1, "value count does not match range"
    sig = DiscreteSignal(start_time, end_time)
    for i, t in enumerate(sig.times()):
        sig.set_value_at_time(t, values[i])
    return sig
```

Some templates instead pad everything into a wide fixed window. Both work; the padded version wastes memory but never overflows a range:

```
def read_signal_padded(filename, INF):
    sig = DiscreteSignal(-INF, INF)
    with open(filename, "r", encoding="utf-8") as f:
        start_time, end_time = map(int, f.readline().split())
        values = list(map(float, f.readline().split()))
    for i, v in enumerate(values):
        sig.set_value_at_time(start_time + i, v)
    return sig
```

**Trap.** With the padded version every output range becomes huge ( $-2*INF \dots 2*INF$ ) and stem plots turn into a forest of zeros. If your plot looks empty, that is why. Use `restrict(sig, a, b)` from Problem 8 to crop before plotting.

### 2.2 Reading several signals from one file

```
def read_blocks(filename):
    """Reads repeated (range line, values line) pairs until the file ends."""
    with open(filename, "r", encoding="utf-8") as f:
        lines = [ln.strip() for ln in f if ln.strip() and not ln.startswith("#")]
    signals, i = [], 0
    while i + 1 < len(lines):
        start_time, end_time = map(int, lines[i].split())
        values = list(map(float, lines[i + 1].split()))
        sig = DiscreteSignal(start_time, end_time)
        for j, t in enumerate(sig.times()):
            sig.set_value_at_time(t, values[j])
        signals.append(sig)
        i += 2
    return signals
```

### 2.3 Building signals inline (no file)

```
# from a list of consecutive values
def make(start_time, end_time, values):
    sig = DiscreteSignal(start_time, end_time)
    for i, t in enumerate(sig.times()):
        sig.set_value_at_time(t, values[i])
    return sig

# from a sparse dictionary
def make_sparse(start_time, end_time, samples):
    sig = DiscreteSignal(start_time, end_time)
    for t, v in samples.items():
        sig.set_value_at_time(t, v)
    return sig

x = make(-2, 3, [1, 0, 2, -1, 0, 3])
d = make_sparse(-5, 5, {0: 1})          # unit impulse
```

### 2.4 Plotting

```
import matplotlib.pyplot as plt

x.plot("Input x[n]")                # one figure, uses the class method
plt.show()                          # <-- WITHOUT THIS NOTHING APPEARS

# several signals in one figure
fig, axes = plt.subplots(3, 1, figsize=(9, 7), constrained_layout=True)
x.plot("Input x[n]", ax=axes[0])
h.plot("Impulse response h[n]", ax=axes[1])
y.plot("Output y[n]", ax=axes[2])
plt.show()

# save instead of showing
fig.savefig("output.png", dpi=150)
```

**Trap.** Every online template plots to screen, not to file. If you forget `plt.show()` the script runs silently and you lose the plotting marks. Put one `plt.show()` as the very last line of `__main__`.

### 2.5 Verification

```
def max_absolute_difference(first, second):
    """Safe comparison: works even when the two signals have different ranges."""
    lo = min(first.start_time, second.start_time)
    hi = max(first.end_time, second.end_time)
    worst = 0.0
    for t in range(lo, hi + 1):
        worst = max(worst, abs(first.get_value_at_time(t) - second.get_value_at_time(t)))
    return worst

diff = max_absolute_difference(y_method_1, y_method_2)
print(f"Maximum absolute difference: {diff:.12g}")
print("Outputs match" if diff < 1e-9 else "Outputs DIFFER")
```

**Trap.** `np.allclose(a.values, b.values)` compares raw arrays. It crashes if the ranges differ in length, and it silently compares the wrong samples if the ranges are shifted. Worse: if your bug is applied to *both* methods (a flipped shift, a missing block) the two agree and the check prints "match" while both answers are wrong. **Always hand-check one sample against the definition.**

2.6 Command-line arguments

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("input_file")
parser.add_argument("--out-dir", default="outputs")
args = parser.parse_args()

import os
os.makedirs(args.out_dir, exist_ok=True)
```

2.7 Idiom table

| You want                             | Write                                                                     |
|--------------------------------------|---------------------------------------------------------------------------|
| <code>x[n-1]</code> (delay by one)   | <code>x.shift(1)</code>                                                   |
| <code>x[n+1]</code> (advance by one) | <code>x.shift(-1)</code>                                                  |
| <code>x[n] - z[n]</code>             | <code>x.add(z.multiply(-1))</code>                                        |
| <code>x[-n]</code> (time reversal)   | Problem 1                                                                 |
| Two systems in series                | <code>LTISystem(h1).output(h2)</code> gives the equivalent <code>h</code> |
| Two systems in parallel              | <code>h1.add(h2)</code>                                                   |
| Gain block of value <code>c</code>   | <code>h.multiply(c)</code>                                                |
| Output length                        | <code>len(x) + len(h) - 1</code>                                          |
| Output range                         | <code>(x.start+h.start, x.end+h.end)</code>                               |
| Unit impulse at <code>n=0</code>     | <code>make(0, 0, [1.0])</code>                                            |
| Feed impulse to a system             | output is <code>h</code> itself — use this to test                        |

**The single most useful fact.** `LTISystem(a).output(b)` convolves `a` with `b`. It does not care which one you call the "system" and which the "signal". That is why series combination, deconvolution setup, correlation and step responses can all be built out of the one method you already wrote.

## Part 3 — Problems and worked solutions

Every solution below was executed; the expected outputs are real program output, not hand-computed. All of them assume the Part 1 classes plus the small helpers `make`, `show` and `max_absolute_difference` from Part 2.

### Problem 1 — Time reversal

Warm-up

**Task.** Add a function `time_reverse(sig)` returning  $y[n] = x[-n]$ . Plot  $x[n]$  and  $y[n]$  in one figure.

**Key idea.** Negating the index flips the window end for end, so the new range is `(-end_time, -start_time)`. Note the swap: the old end becomes the new start.

```
def time_reverse(sig):
    out = DiscreteSignal(-sig.end_time, -sig.start_time)
    for t in sig.times():
        out.set_value_at_time(-t, sig.get_value_at_time(t))
    return out

# one-line alternative:
# out = DiscreteSignal(-sig.end_time, -sig.start_time)
# out.values = sig.values[::-1].copy()
```

Expected output.

```
x over -1..2 = [1, 2, 3, 4]
time_reverse(x) -> [(-2, 4.0), (-1, 3.0), (0, 2.0), (1, 1.0)]
```

Reversal is its own inverse: `time_reverse(time_reverse(x))` must equal `x`. Free self-check if you have a spare minute.

### Problem 2 — Subtract and delay

Warm-up

**Task.** Add `subtract(a, b)` and `delay(sig, N)` giving  $y[n] = x[n-N]$ . You must build them out of existing Signal operations only.

**Key idea.** There is no subtract method, so scale by -1 and add. Delay by N is exactly `shift(N)` — positive N moves the signal to the right.

```
def subtract(first, second):
    return first.add(second.multiply(-1))

def delay(sig, n_samples):
    return sig.shift(n_samples)
```

Expected output.

```
delay(make(0,3,[1,2,3,4]), 2) -> [(2, 1.0), (3, 2.0), (4, 3.0), (5, 4.0)]
subtract(make(0,2,[5,5,5]), make(1,3,[1,2,3])) -> [(0, 5.0), (1, 4.0), (2, 3.0), (3, -3.0)]
```

The subtract result spans 0..3, wider than either input. `add` handles the range union for you — never assume the result has the same range as the first argument.

### Problem 3 — Even and odd decomposition

Warm-up

**Task.** Split  $x[n]$  into its even and odd parts, where  $x_e[n] = (x[n] + x[-n])/2$  and  $x_o[n] = (x[n] - x[-n])/2$ . Verify that the two parts add back to  $x[n]$ .

**Key idea.** Build it directly from the formula using `time_reverse` from Problem 1. Both parts end up on the symmetric range `(-max, +max)` because add unions the ranges.

```
def even_odd_parts(sig):
    reversed_sig = time_reverse(sig)
    even_part = sig.add(reversed_sig).multiply(0.5)
    odd_part = subtract(sig, reversed_sig).multiply(0.5)
    return even_part, odd_part

even_part, odd_part = even_odd_parts(x)
print("reconstruction error:", max_absolute_difference(even_part.add(odd_part), x))
```

Expected output.

```
x over 0..2 = [4, 2, 0]
even -> [(-2, 0.0), (-1, 1.0), (0, 4.0), (1, 1.0), (2, 0.0)]
odd -> [(-2, 0.0), (-1, -1.0), (0, 0.0), (1, 1.0), (2, 0.0)]
reconstruction error: 0.0
```

Sanity check without running anything: the even part must be symmetric about  $n = 0$  and the odd part must have  $x_o[0] = 0$ . Both hold above.

### Problem 4 — Energy and average power

Warm-up

**Task.** Add `energy(sig)` and `average_power(sig)`, then report both for the input and the output of a system.

**Key idea.** Energy is the sum of squared samples. Since the signal is finite you can use the NumPy array directly rather than looping.

```
def energy(sig):
    return float(np.sum(sig.values ** 2))

def average_power(sig):
    return energy(sig) / len(sig)
```

Expected output.

```
x over 0..3 = [1, -2, 3, 0]
energy   = 14.0
average_power = 3.5
```

If the question asks you to show a system is *passive* or *lossless*, compare energy(y) against energy(x). A moving-average filter loses energy; the identity keeps it.

## Problem 5 — Upsampling and downsampling

Warm-up

**Task.** Add `upsample(sig, L)` which inserts L-1 zeros between samples, and `downsample(sig, M)` which keeps every M-th sample. Plot both.

**Key idea.** Upsampling multiplies every time index by L, so the range scales too. Downsampling keeps  $y[n] = x[nM]$ , so the new range is `ceil(start/M) .. floor(end/M)` — use `math.ceil/math.floor`, not integer division, because Python rounds negative division towards minus infinity.

```
import math

def upsample(sig, factor):
    out = DiscreteSignal(sig.start_time * factor, sig.end_time * factor)
    for t in sig.times():
        out.set_value_at_time(t * factor, sig.get_value_at_time(t))
    return out

def downsample(sig, factor):
    start = math.ceil(sig.start_time / factor)
    end   = math.floor(sig.end_time / factor)
    out = DiscreteSignal(start, end)
    for n in out.times():
        out.set_value_at_time(n, sig.get_value_at_time(n * factor))
    return out
```

**Expected output.**

```
upsample(make(0,2,[1,2,3]), 3)
-> [(0,1.0), (1,0.0), (2,0.0), (3,2.0), (4,0.0), (5,0.0), (6,3.0)]
downsample(make(-3,4,[7,1,8,2,9,3,6,4]), 2)
-> [(-1,1.0), (0,2.0), (1,3.0), (2,4.0)]
```

Downsampling is the one operation in this whole pack that is **not** time-invariant. If the question asks you to justify why, that is the answer: delaying the input by one sample does not simply delay the output by one sample.

## Problem 6 — Two systems in series (cascade)

Core

**Task.** Two LTI systems with impulse responses h1 and h2 are connected in series. Compute the output by passing x through h1 then h2, then compute the single equivalent impulse response  $h_{eq}$  and apply it directly. Verify the two outputs match.

**Key idea.** Series connection convolves the impulse responses. Since `LTISystem(a).output(b)` is just convolution, feeding h2 into a system built from h1 gives  $h_{eq}$  — no new code required.

```
def cascade(h_first, h_second):
    return LTISystem(h_first).output(h_second)

# stage by stage
stage_1 = LTISystem(h1).output(x)
y_blocks = LTISystem(h2).output(stage_1)

# equivalent single system
h_eq = cascade(h1, h2)
y_combined = LTISystem(h_eq).output(x)

print("difference:", max_absolute_difference(y_blocks, y_combined))
```

**Expected output.**

```
h1 = [1, 1] over 0..1, h2 = [1, -1] over 0..1, x = [1, 2, 3] over 0..2
h_eq -> [(0, 1.0), (1, 0.0), (2, -1.0)]
difference: 0.0
```

Order does not matter: `cascade(h1, h2)` equals `cascade(h2, h1)`. If a question asks whether swapping two blocks changes the output, the answer is no, and commutativity of convolution is the reason.

## Problem 7 — Mixed block diagram

Core

**Task.** x feeds h1. The output of h1 splits into two parallel branches h2 and h3, whose outputs are summed and passed through h4 to give y. Compute y block by block, then find the single equivalent impulse response and verify.

**Key idea.** Translate the picture one connector at a time. Parallel means `add`. Series means `convolve`. So  $h_{eq} = h1 * (h2 + h3) * h4$ .



```
def network_equivalent(h1, h2, h3, h4):
    middle = h2.add(h3)          # parallel branches
    return cascade(cascade(h1, middle), h4) # series stages

# block by block, following the diagram exactly
after_1 = LTISystem(h1).output(x)
branch_2 = LTISystem(h2).output(after_1)
branch_3 = LTISystem(h3).output(after_1)
y_blocks = LTISystem(h4).output(branch_2.add(branch_3))

# single equivalent system
y_combined = LTISystem(network_equivalent(h1, h2, h3, h4)).output(x)
print("difference:", max_absolute_difference(y_blocks, y_combined))
```

#### Expected output.

```
h1 = [1] at 0, h2 = [0, 0.5] over 0..1, h3 = [1, 1] over 0..1, h4 = [1, -1] over 0..1
h_eq -> [(0, 1.0), (1, 0.5), (2, -1.5)]
```

A gain block labelled with a plain number  $c$  is `h.multiply(c)`, and a pure delay of  $N$  is `h.shift(N)`. With add, multiply, shift and output you can encode any block diagram they can draw.

## Problem 8 — Step response and impulse response

Core

**Task.** You are given the step response  $s[n]$  of a system. Recover the impulse response  $h[n] = s[n] - s[n-1]$ , then go the other way and rebuild  $s[n]$  from  $h[n]$  as a running sum. Verify both directions.

**Key idea.** Differencing and accumulation are inverses. The catch is range bookkeeping: `first_difference` extends the range by one sample at the right-hand end, and that extra sample is an artefact of  $s$  being stored on a finite window, not a real part of  $h$ . Crop it away.

```
def first_difference(sig):
    return subtract(sig, sig.shift(1))      # sig[n] - sig[n-1]

def restrict(sig, start_time, end_time):
    out = DiscreteSignal(start_time, end_time)
    for n in out.times():
        out.set_value_at_time(n, sig.get_value_at_time(n))
    return out

def impulse_from_step_response(s):
    return restrict(first_difference(s), s.start_time, s.end_time)

def step_response_from_impulse(h, extra=0):
    out = DiscreteSignal(h.start_time, h.end_time + extra)
    running = 0.0
    for n in out.times():
        running += h.get_value_at_time(n)
        out.set_value_at_time(n, running)
    return out
```

#### Expected output.

```
h over 0..3 = [1, -1, 2, 0.5]
step_response_from_impulse(h) -> [(0,1.0), (1,0.0), (2,2.0), (3,2.5)]
first_difference(s)           -> [(0,1.0), (1,-1.0), (2,2.0), (3,0.5), (4,-2.5)]  <-- spurious tail
impulse_from_step_response(s) -> [(0,1.0), (1,-1.0), (2,2.0), (3,0.5)]           <-- correct
difference from original h: 0.0
```

This is the July 2025 A1/A2 paper. It also asks for  $y[n]$  using only  $s[n]$ , via  $y = \Delta x * s$ . That works because convolution commutes with differencing:  $(\Delta x) * s = x * (\Delta s) = x * h$ . Write it as `LTISystem(s).output(first_difference(x))` and verify against `LTISystem(h).output(x)`.

## Problem 9 — Cross-correlation

Core

**Task.** Add a function computing the cross-correlation  $r[n] = \sum_k x[k] y[k-n]$  without writing a new double loop.

**Key idea.** Correlation is convolution with one signal time-reversed. Substituting  $m = n-k$  shows  $r[n] = (x * y_{\text{rev}})[n]$  where  $y_{\text{rev}}[m] = y[-m]$ . So reuse Problem 1 plus your existing output method.

```
def cross_correlate(x, y):
    return LTISystem(time_reverse(y)).output(x)
```

#### Expected output.

```
x over 0..5 = [0, 0, 1, 2, 1, 0], y over 0..2 = [1, 2, 1]
r -> [(-2,0.0), (-1,0.0), (0,1.0), (1,4.0), (2,6.0), (3,4.0), (4,1.0), (5,0.0)]
```

The result range is `(x.start - y.end, x.end - y.start)`. Autocorrelation is `cross_correlate(x, x)` and always peaks at lag 0 — another free self-check.

## Problem 10 — Causality, memory and stability

Core

**Task.** Given an arbitrary impulse response, report whether the system is causal, whether it is memoryless, and whether it is BIBO stable.

**Key idea.** Causal means  $h[n] = 0$  for all  $n < 0$ . Memoryless means only  $h[0]$  may be nonzero. BIBO stability means  $\sum |h[n]|$  is finite, which for a finite-length  $h$  is automatic — so report the actual sum, which is what earns the mark.

```
def is_causal(h, tolerance=1e-12):
    for n in h.times():
        if n < 0 and abs(h.get_value_at_time(n)) > tolerance:
            return False
    return True

def is_memoryless(h, tolerance=1e-12):
    for n in h.times():
        if n != 0 and abs(h.get_value_at_time(n)) > tolerance:
            return False
    return True

def absolute_sum(h):
    return float(np.sum(np.abs(h.values)))
```

#### Expected output.

```
h over 0..2 = [1, 2, 3]      -> causal True,  memoryless False, abs_sum 6.0
h over -1..1 = [1, 2, 3]    -> causal False
h over 0..0 = [5]           -> memoryless True
```

Test on h's stored range, not on a padded window: a signal built over `(-INF, INF)` has explicit zeros at negative indices, which your loop must tolerate. The tolerance comparison handles that.

### Problem 11 — Deconvolution — recover the input

Stretch

**Task.** Given the output  $y[n]$  and the impulse response  $h[n]$  with  $h[h_{\text{start}}] \neq 0$ , recover the input  $x[n]$ . Verify by convolving your answer back with  $h$ .

**Key idea.** Solve the convolution sum one sample at a time. At the earliest output index only one unknown appears, so you can divide it out; each later sample subtracts the already-known contributions and divides by the leading tap. This is polynomial long division.

```
def deconvolve(y, h):
    h_start = h.start_time
    leading = h.get_value_at_time(h_start)
    if abs(leading) < 1e-12:
        raise ValueError("First impulse-response sample must be nonzero")

    x_start = y.start_time - h_start
    x_end = y.end_time - h.end_time
    x = DiscreteSignal(x_start, x_end)

    for n in x.times():
        known = 0.0
        for k in range(x_start, n):
            known += x.get_value_at_time(k) * h.get_value_at_time(n + h_start - k)
        x.set_value_at_time(n, (y.get_value_at_time(n + h_start) - known) / leading)
    return x
```

#### Expected output.

```
x over -1..2 = [1, 2, -1, 3],  h over 0..2 = [1, 0.5, -0.25]
y  -> [(-1,1.0), (0,2.5), (1,-0.25), (2,2.0), (3,1.75), (4,-0.75)]
recovered x -> [(-1,1.0), (0,2.0), (1,-1.0), (2,3.0)]
difference from true x: 0.0
```

The derivation in one line:  $y[n + h_{\text{start}}] = \sum_{k < n} x[k] h[n + h_{\text{start}} - k] + x[n] h[h_{\text{start}}]$ , because every  $k > n$  needs an  $h$  index below  $h_{\text{start}}$ , which is zero.

### Problem 12 — System identification — recover h

Stretch

**Task.** Given an input  $x[n]$  and the observed output  $y[n]$ , find the impulse response of the unknown system.

**Key idea.** Convolution is commutative, so recovering  $h$  from  $(y, x)$  is the same algorithm as recovering  $x$  from  $(y, h)$ . Swap the arguments — no new code at all.

```
h_recovered = deconvolve(y, x)      # same function as Problem 11, arguments swapped

check = LTISystem(h_recovered).output(x)
print("residual:", max_absolute_difference(check, y))
```

#### Expected output.

```
same x and y as Problem 11
h_recovered -> [(0, 1.0), (1, 0.5), (2, -0.25)]
difference from true h: 0.0
```

If they instead let you choose the input, choose the unit impulse: feed `make(0,0,[1.0])` into the system and the output is  $h$ . Mention that in a comment even if you also write the deconvolution — it shows you understand the definition.

### Problem 13 — SuperSignal

Stretch

**Task.** Write a class `SuperSignal` that stores several signals with scalar coefficients. Add a method `output_super` to the LTI class taking a `SuperSignal` and returning the output. Verify against collapsing the `SuperSignal` into one signal first.

**Key idea.** This is linearity made concrete: the response to  $\sum c_i x_i$  equals  $\sum c_i (\text{response to } x_i)$ . Accumulate with `result = result.add(...)` — add returns a new object and does not mutate.

```

class SuperSignal:
    def __init__(self):
        self.components = []

    def add(self, signal, coefficient=1.0):
        self.components.append((coefficient, signal))

# inside LTISystem:
def output_super(self, super_signal):
    result = None
    for coefficient, signal in super_signal.components:
        piece = self.output(signal).multiply(coefficient)
        result = piece if result is None else result.add(piece)
    return result if result is not None else DiscreteSignal(0, 0)

# the other route, for verification
def collapse_super(super_signal):
    result = None
    for coefficient, signal in super_signal.components:
        piece = signal.multiply(coefficient)
        result = piece if result is None else result.add(piece)
    return result

```

#### Expected output.

```

x1 = impulse at 0, x2 = impulse at 2, super = 2*x1 - 1*x2, h = [1, 0.5] over 0..1
output_super -> [(0,2.0), (1,1.0), (2,-1.0), (3,-0.5)]
difference against collapse-then-convolve: 0.0

```

This is the July 2025 C1/C2 paper. The template's main block writes `x2.multiply(-1)` on its own line, which discards the result — the whole point of the coefficient argument is that you write `ss.add(x2, -1.0)` instead.

### Problem 14 — General system network

Stretch

**Task.** Write a class that stores any number of parallel branches, each branch being a series chain of impulse responses with an optional gain, and returns the single equivalent impulse response.

**Key idea.** Fold each chain with convolution, scale by the gain, then fold the branches with add. Two nested accumulations, each using the `result = result.op(...)` pattern.

```

class SystemNetwork:
    def __init__(self):
        self.branches = []

    def add_branch(self, impulse_responses, gain=1.0):
        self.branches.append((gain, list(impulse_responses)))

    def equivalent_impulse_response(self):
        total = None
        for gain, chain in self.branches:
            h = chain[0]
            for next_h in chain[1:]:
                h = LTISystem(h).output(next_h) # series
            h = h.multiply(gain)
            total = h if total is None else total.add(h) # parallel
        return total

    def output(self, input_signal):
        return LTISystem(self.equivalent_impulse_response()).output(input_signal)

```

#### Expected output.

```

branch 1 = [hA, hC] with hA = [1] at 0 and hC = [1, 1] over 0..1
branch 2 = [hB] with hB = [0, 0.5] over 0..1, gain -1
h_eq -> [(0, 1.0), (1, 0.5)]
output for x = [1, 1] over 0..1 -> [(0,1.0), (1,1.5), (2,0.5)]

```

Handles Problem 6 and Problem 7 as special cases. If the exam draws a diagram with more than three blocks, writing this class is faster and less error-prone than tracing each connector by hand.

### Problem 15 — Circular convolution

Stretch

**Task.** Implement N-point circular convolution and demonstrate when it agrees with ordinary linear convolution and when it does not.

**Key idea.** Index arithmetic modulo N wraps the tail of the linear result back onto the front. The two agree exactly when N is at least  $\text{len}(x) + \text{len}(h) - 1$ ; below that you get time-domain aliasing.

```

def circular_convolution(x, h, period):
    out = DiscreteSignal(0, period - 1)
    for n in range(period):
        acc = 0.0
        for m in range(period):
            acc += x.get_value_at_time(m) * h.get_value_at_time((n - m) % period)
        out.set_value_at_time(n, acc)
    return out

```

#### Expected output.

```

x = [1, 2] over 0..1, h = [1, 1] over 0..1 (linear result needs N >= 3)
linear convolution   -> [(0,1.0), (1,3.0), (2,2.0)]
circular with N = 3 -> [(0,1.0), (1,3.0), (2,2.0)] matches
circular with N = 2 -> [(0,3.0), (1,3.0)] aliased: 2.0 wrapped onto n=0

```

Both signals must be indexed from 0 for the modulo arithmetic to mean anything. If the question gives you signals starting elsewhere, shift them to start at 0 first and note the shift in a comment.

## Problem 16 — Recursive system versus truncated FIR

Stretch

**Task.** Implement the recursive system  $y[n] = a \cdot y[n-1] + x[n]$  directly, then show its impulse response is  $h[n] = a^n u[n]$  and that convolving with a long-enough truncation of  $h$  reproduces the same output.

**Key idea.** The recursion carries state across the loop, so it cannot be written with shift and add. Convolution needs a finite  $h$ , so truncate at a length where  $a^n$  is negligible — then the two agree over the range you care about.

```
def first_order_recursive(x, a, end_time):
    out = DiscreteSignal(x.start_time, end_time)
    previous = 0.0
    for n in out.times():
        previous = a * previous + x.get_value_at_time(n)
        out.set_value_at_time(n, previous)
    return out

def truncated_exponential(a, length):
    out = DiscreteSignal(0, length - 1)
    for n in out.times():
        out.set_value_at_time(n, a ** n)
    return out
```

**Expected output.**

```
a = 0.5, impulse input
recursive output      -> [(0,1.0), (1,0.5), (2,0.25), (3,0.125), (4,0.0625), (5,0.0312)]
truncated_exponential -> [(0,1.0), (1,0.5), (2,0.25), (3,0.125), (4,0.0625), (5,0.0312)]
for x = [1,1,1,1], max difference over n = 0..8 using a 12-tap truncation: 0.0
```

Stability shows up here concretely. With  $|a| < 1$  the impulse response decays and  $\sum |h[n]|$  converges, so the system is BIBO stable. With  $|a| \geq 1$  it grows without bound and no finite truncation is honest — say so rather than picking a bigger length.

## Problem 17 — Pattern detection with a matched filter

Stretch

**Task.** Given a long signal and a short pattern, find the lag at which the pattern best matches, using correlation.

**Key idea.** Cross-correlate and take the argmax. With  $r[n] = \sum_k x[k]y[k-n]$ , the peak lands at the index where the pattern starts inside the long signal.

```
def best_match_lag(long_signal, pattern):
    scores = cross_correlate(long_signal, pattern)
    best_lag = scores.start_time
    best_score = scores.get_value_at_time(best_lag)
    for n in scores.times():
        if scores.get_value_at_time(n) > best_score:
            best_score = scores.get_value_at_time(n)
            best_lag = n
    return best_lag, float(best_score)
```

**Expected output.**

```
long_signal over 0..9 = [0, 0, 1, 3, 1, 0, 0, 1, 0, 0], pattern over 0..2 = [1, 3, 1]
scores -> [(0,1.0), (1,6.0), (2,11.0), (3,6.0), (4,1.0), (5,1.0), (6,3.0), (7,1.0), ...]
best lag = 2, score = 11.0      (the pattern occupies indices 2, 3, 4)
```

Raw correlation favours loud regions over well-shaped ones. If the question mentions noise or unequal amplitudes, divide each score by the local energy of the window — that is normalised cross-correlation, and mentioning it is usually worth a mark.

## Part 4 — Application problems (January 2024 papers)

These papers are a different flavour from the 2025/2026 set. Instead of combining systems, they describe a real-world computation and make you work out what  $x$  and  $h$  should be. All three follow one recipe:

1.  $x[k]$  is the data sequence, indexed from 0.
2.  $h[i]$  is the weight applied to the sample  $i$  steps in the past, so  $h[0]$  multiplies the *most recent* sample.
3. Run the full convolution, then keep only  $t = n-1 \dots m-1$ , which is exactly  $m-n+1$  values.

**The reversal trap.** Question tables draw the weight window left to right as oldest  $\rightarrow$  newest. Your  $h$  array is the **reverse** of that picture, because in  $y[t] = \sum_k x[k]h[t-k]$  the newest sample ( $k = t$ ) pairs with  $h[0]$ . A symmetric kernel like the unweighted average hides the mistake completely — so if your unweighted answer is right and your weighted answer is wrong, this is why.

### Shared helpers

```
def signal_from_list(values, start_time=0):
    sig = DiscreteSignal(start_time, start_time + len(values) - 1)
    for i, v in enumerate(values):
        sig.set_value_at_time(start_time + i, v)
    return sig

def window_slice(y, window_size, data_length):
    """Keep only the outputs where the window sits fully inside the data."""
    return [y.get_value_at_time(t) for t in range(window_size - 1, data_length)]

def fmt(values, places=2):
    return ", ".join(f"{v:.{places}f}" for v in values)
```

### Paper A (Jan 2024) — Exponential smoothing

**Task.** Smooth stock prices with  $h[k] = \alpha(1-\alpha)^k$  for  $k = 0 \dots n-1$ . Print  $m-n+1$  values.

```
def exponential_kernel(window_size, alpha):
    return signal_from_list([alpha * (1 - alpha) ** k for k in range(window_size)])

def exponential_smoothing(prices, window_size, alpha):
    x = signal_from_list(prices)
    y = LTISystem(exponential_kernel(window_size, alpha)).output(x)
    return window_slice(y, window_size, len(prices))

raw = input("Stock Prices: ")
prices = [float(v) for v in raw.replace(",", " ").split()]
window_size = int(input("Window size: "))
alpha = float(input("Alpha: "))
print(fmt(exponential_smoothing(prices, window_size, alpha)))
```

```
prices = 10,11,12,9,10,13,15,16,17,18    n = 3    alpha = 0.8
kernel h -> [0.8, 0.16, 0.032]
output    -> 11.68, 9.47, 9.82, 12.29, 14.40, 15.62, 16.64, 17.63    (matches the paper)
```

This kernel is **not** normalised — it sums to  $1-(1-\alpha)^n = 0.992$ , not 1. The paper does not ask you to normalise and doing so breaks the sample output. Do not "fix" it.

### Paper B (Jan 2024) — Unweighted and weighted moving average

**Task.** Same prices, window  $n$ . Unweighted gives every day weight  $1/n$ . Weighted gives the newest day weight  $n$ , the one before  $n-1$ , and so on, normalised to sum to 1.

```
def unweighted_kernel(window_size):
    return signal_from_list([1.0 / window_size] * window_size)

def weighted_kernel(window_size):
    total = window_size * (window_size + 1) / 2          # 1 + 2 + ... + n
    return signal_from_list([(window_size - i) / total for i in range(window_size)])

def moving_average(prices, window_size, kernel):
    x = signal_from_list(prices)
    y = LTISystem(kernel).output(x)
    return window_slice(y, window_size, len(prices))

raw = input("Stock Prices: ")
prices = [float(v) for v in raw.replace(",", " ").split()]
n = int(input("Window size: "))
print("Unweighted Moving Average:", fmt(moving_average(prices, n, unweighted_kernel(n))))
print("Weighted Moving Average:",    fmt(moving_average(prices, n, weighted_kernel(n))))
```

```
prices = 1 2 3 4 5 6 7 8    n = 4
unweighted kernel -> [0.25, 0.25, 0.25, 0.25]
weighted   kernel -> [0.4, 0.3, 0.2, 0.1]    <-- reverse of the table in the question
Unweighted Moving Average: 2.50, 3.50, 4.50, 5.50, 6.50
Weighted Moving Average: 3.00, 4.00, 5.00, 6.00, 7.00

prices = 5 -2 3 1 0 -6 4 -2 1    n = 3
Unweighted: 2.00, 0.67, 1.33, -1.67, -0.67, -1.33, 1.00
Weighted:    1.67, 1.17, 0.83, -2.83, 0.00, -0.67, 0.50    (both match the paper)
```

### Paper C (Jan 2024) — Polynomial multiplication

**Task.** Multiply two polynomials given by degree plus coefficients in decreasing exponent order.

```
def multiply_polynomials(coeff_a, coeff_b):
    x = signal_from_list(coeff_a)          # coefficients exactly as given
    h = signal_from_list(coeff_b)
    y = LTISystem(h).output(x)
    return [y.get_value_at_time(t) for t in y.times()]

d1 = int(input("Degree of the first Polynomial: "))
a = [int(v) for v in input("Coefficients: ").split()]
d2 = int(input("Degree of the second Polynomial: "))
b = [int(v) for v in input("Coefficients: ").split()]

result = [int(round(v)) for v in multiply_polynomials(a, b)]
print("Degree of the Polynomial:", len(result) - 1)
print("Coefficients:", " ".join(str(v) for v in result))
```

```
(3 -2 1) x (2 0 -3 1)      -> degree 5, coefficients 6 -4 -7 9 -5 1
(1 2 -1) x (2 -1 0 3 2)    -> degree 6, coefficients 2 3 -4 4 8 1 -2
(3 -2 0 1 -5) x (1 2 -3 -1)-> degree 7, coefficients 3 4 -13 4 -1 -13 14 5
all three match the paper
```

No reversal and no trimming here — keep the whole output. If  $a[k]$  is the coefficient of  $x^{d1-k}$  and  $b[m]$  of  $x^{d2-m}$ , their product lands on  $x^{d1+d2-k-m}$ , which is output index  $k+m$ . Summing over  $k+m = t$  is exactly  $c[t] = \sum_k a[k]b[t-k]$ . The result degree is  $d1+d2$ , so the coefficient count is  $d1+d2+1$  — the familiar  $\text{len}(x)+\text{len}(h)-1$ .

## Part 5 — Pre-submission trap checklist

Two minutes on this list has caught a real bug in every past paper reviewed.

| #  | Check                                                                                                         | Why it bites                                                                                                                             |
|----|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Is <code>x[n-1]</code> written as <code>shift(1)</code> ?                                                     | <code>shift(-1)</code> gives <code>x[n+1]</code> . Sign-flips the whole answer, and the verification still passes.                       |
| 2  | Did you assign the result of every <code>add</code> / <code>multiply</code> / <code>shift</code> ?            | They return new objects and never mutate. <code>res.add(y)</code> alone is a no-op.                                                      |
| 3  | Are you unpacking tuples where the container stores pairs?                                                    | SuperSignal stores <code>(coefficient, signal)</code> . Passing the tuple gives <code>AttributeError</code> on <code>start_time</code> . |
| 4  | Did every block in the diagram get used?                                                                      | Dropping the last stage makes both computation routes wrong identically, so they still agree.                                            |
| 5  | Is <code>plt.show()</code> the last line of <code>main</code> ?                                               | Without it the script runs silently and produces no figures.                                                                             |
| 6  | Are you accumulating inside the loop, not after it?                                                           | An <code>accumulate</code> line at the wrong indentation keeps only the last term.                                                       |
| 7  | Does a function use its parameter rather than a same-named global?                                            | Works by luck in <code>main</code> , fails the grader's test harness.                                                                    |
| 8  | Did you compare with <code>max_absolute_difference</code> rather than <code>np.allclose</code> on raw arrays? | Raw arrays crash or silently compare misaligned samples when ranges differ.                                                              |
| 9  | Does the output length equal <code>len(x) + len(h) - 1</code> ?                                               | Fastest possible check that your output range is right.                                                                                  |
| 10 | For a weighted window, is <code>h</code> reversed relative to the picture in the question?                    | <code>h[0]</code> multiplies the newest sample. A symmetric kernel hides the error entirely.                                             |
| 11 | Did you hand-verify one single output sample against the convolution sum?                                     | The only check that catches a symmetric bug in both methods.                                                                             |

### The thirty-second sanity test

Whatever the question, before you submit, feed a unit impulse into your system:

```
impulse = make(0, 0, [1.0])
print(show(LTISystem(h).output(impulse)))    # must reproduce h exactly
```

If that does not return `h` unchanged, something in your convolution, shifting or range arithmetic is broken and no other answer on the page will be right.